

How to Build the Prediction Engine

A plain-language guide to the logic, the code, and the order in which to build it from scratch.

CONTENTS

1	What we are building and why	2
2	The learning loop	3
3	Starting with a guess — the baseline model	4
4	Learning from your data — the Bayesian model	5
5	The five signals the model uses	6
6	Keeping data clean — the validator	7
7	How a prediction request travels through the system	8
8	The cloud plumbing — AWS services	9
9	The offline version	10
10	Build order — how to approach this from scratch	11

Project	Organisation	Module	Date
Oky Period Tracker	UNICEF	New Prediction Engine	June 2026

SECTION 1

What we are building and why

Oky is a period-tracking app used by 500,000+ adolescent girls across 35+ countries. The app needs to tell each user when her next period is likely to start — and get more accurate over time as it learns her personal pattern.

The core problem in one sentence: On day one we know almost nothing about this specific user, so we have to make a reasonable guess. After a few months of data, the guess should become a personal prediction.

Three things the engine needs to do

1 Make a prediction

Given a user's cycle history, predict when her next period will likely start — with a range showing how confident the prediction is.

2 Learn from confirmed cycles

Every time a user confirms that a period started, that information is fed back into the model so the next prediction is more accurate.

3 Work offline

The app must work without internet. The same prediction logic runs locally on the device, and syncs to the cloud when connection is restored.

Two models, not one

BASELINE MODEL

Used for new users (fewer than 3 confirmed cycles). Starts from population data. Blends in the user's personal data as it arrives. Simple, fast, predictable.

BAYESIAN MODEL

Takes over from cycle 4 onward. Remembers everything it has learned about this user and updates its internal beliefs with each new data point. Gets better the more data it has.

Why two models? The Bayesian model needs at least a few data points to be reliable. The baseline model gives a reasonable answer while the Bayesian model is still "warming up."

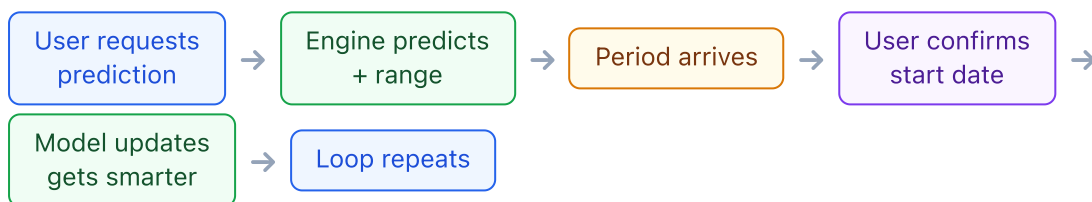
SECTION 2

The learning loop

Most software returns the same answer to the same question. A prediction engine is different: the same question asked two months later should return a better answer, because the system has learned from what happened in between.

The analogy: Think of a doctor estimating a patient's resting heart rate. On the first visit, the doctor uses the population average: "Most adults are around 70 bpm." After five visits with measurements of 58, 61, 59, 57, 60 — the doctor says "your resting rate is about 59 bpm." Same person, much better estimate.

The loop in steps



Prediction and learning are *separate* operations. Predicting is read-only and fast. Learning (updating) runs in the background through a queue so it does not slow down the app.

The transition point

After exactly **3 confirmed cycles**, the engine switches from the baseline model to the Bayesian model. This threshold is a deliberate design choice: 3 data points are the minimum needed for the Bayesian model to have a meaningful advantage over the population average.

Prediction is always instant. The engine stores the learned model state in a database. When a prediction is requested, it loads that state and runs the model in milliseconds. No recomputation from scratch, no waiting for history to be replayed.

SECTION 3

Starting with a guess — the baseline model

Before a user has tracked any cycles, the best we can do is use what we know about the population. Research tells us that most people's cycles are close to 28 days, with most falling between about 25 and 31 days.

What the baseline model stores

STARTING VALUES (CONFIG)

```
default_cycle_length = 28.0
```

The starting guess for everyone.

```
mature_user_cycle_std = 3.0
```

Uncertainty window for 2+ years of periods. Roughly ± 3 days $\rightarrow$ range 25–31.

```
young_user_cycle_std = 5.0
```

Wider for first 2 years. Cycles are more variable early on.

WHAT GETS RETURNED

A prediction like: "Next period expected in 28 days, likely somewhere between 22 and 34 days." That range comes from the standard deviation above. Confidence is fixed at 0.5 — honest about how uncertain we are with no personal data.

Blending in personal data

As the user confirms cycles, the engine blends their personal data gradually with the population average. This avoids overreacting to a single unusual data point.

0 cycles logged

100% population average (28 days)

1 cycle logged

33% yours

67% population

2 cycles logged

50% yours

50% population

3+ cycles $\rightarrow$ Bayesian takes over

Fully personalised — Bayesian model

The blending formula is: `weight = min(number_of_cycles / 3.0, 0.5)`. Your personal average is multiplied by this weight; the population average gets the remainder. The cap at 0.5 ensures

the baseline never pretends to know more than it does.

Why cap at 0.5? After 2 cycles you have just two data points — not enough to trust your personal pattern more than the global average. The cap forces a clean transition to the Bayesian model at cycle 3 rather than letting the baseline drift indefinitely.

SECTION 4

Learning from your data — the Bayesian model

The word "Bayesian" sounds complicated but the idea is simple: **start with a belief, then update it every time you see new evidence.** The update rule is mathematically precise — it tells you exactly how much to shift your belief based on how surprising the new evidence was.

The Bayesian idea in everyday terms: You believe a coin is fair (50/50). You flip it 10 times and get 7 heads. You update slightly — maybe 55/45 in favour of heads. You flip 100 more times and get 70 heads. You update more strongly — maybe 65/35. You never fully commit until you have enough evidence, but you always move in the right direction.

What the model stores about each user

The Bayesian model does not just store the latest prediction. It stores a complete picture of its own uncertainty. Think of it as storing both a best guess and a confidence level for that guess.

w Weights (mean_w) — 5 numbers

Together these define the relationship between this user's cycle history and her next predicted cycle. Think of them as five "dials" that the model adjusts based on her data. They start at neutral values and drift toward her personal pattern.

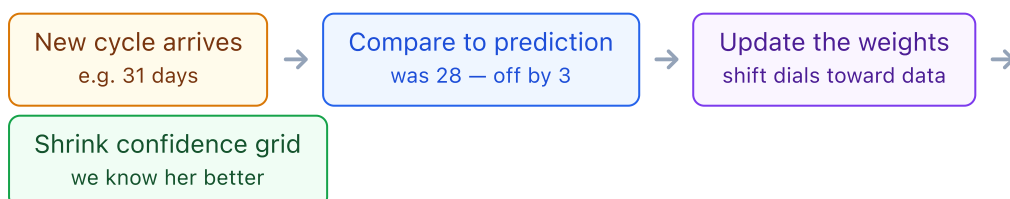
c Confidence (cov_w) — a 5×5 grid

Captures how certain the model is about each of the 5 weights and how they relate to each other. Large values = uncertain. Small values = confident. This grid shrinks with each confirmed cycle, producing a narrower prediction range over time.

n Sample count (n_samples)

How many confirmed cycles has this user contributed? Used to decide whether to use baseline or Bayesian (threshold: 3 cycles).

What happens when a new cycle is confirmed



The more consistent the cycles, the tighter the range. If a user's last 12 cycles were all 27–29 days, the confidence grid becomes very small and the predicted range narrows to perhaps ± 1.5 days. If cycles are erratic (22 to 36), the grid stays large and the range stays wide — which is an honest representation of the uncertainty.

SECTION 5

The five signals the model uses

The Bayesian model does not just look at your most recent cycle and repeat that number. It looks at five different things about your cycle history and combines them into one prediction. These are called *features* — think of them as five ingredients in a recipe, each contributing something different to the result.

SIGNAL	WHAT IT CAPTURES	EXAMPLE
recent mean	The average length of your last 3 cycles.	Cycles were 27, 29, 28 → average = 28. The model leans most heavily on this.
recent std	How much your last 3 cycles varied from each other.	27, 29, 28 → low variation (≈ 1). 22, 35, 26 → high variation (≈ 7). Higher variation = wider predicted range.
trend	Are cycles slowly getting longer or shorter? Only available after 6+ confirmed cycles.	Cycles: 26, 27, 27, 28, 28, 29 → slight upward trend. The model adds a small amount to the next prediction.
regularity	How regular is this user overall? A score from 0 to 1.	Consistent 28-day user scores ≈ 1.0 . A user with 22, 36, 25, 40 scores much lower.
bias (constant)	A baseline offset — always 1.0. Lets the model learn a personal offset from the population average.	If everything else predicts 28 but this user consistently runs 2 days longer, the bias learns and adds 2.

How these combine: Each signal is multiplied by its learned weight (one of the 5 "dials" from Section 4) and the results are added together. The final sum is the predicted cycle length. All 5 dials are adjusted simultaneously every time a new confirmed cycle comes in.

Trend only activates after 6 confirmed cycles. You cannot reliably detect a trend from 2–3 data points — it might just be noise. The model waits until there is enough history, and uses 0 for the trend signal until then. Before 6 cycles, this feature simply contributes nothing.

Where this lives in the code

```

# bayesian_model.py - building the feature vector for one prediction
recent_cycles = cycle_history[-3:]          # last 3 cycles
recent_mean   = np.mean(recent_cycles)      # signal 1
recent_std    = np.std(recent_cycles)       # signal 2
regularity    = 1.0 / (1.0 + recent_std / 3.0) # signal 4 (derived)
trend         = compute_trend(cycle_history) # signal 3 (0 if < 6 cycles)
features      = [1.0, recent_mean, recent_std, regularity, trend] # signal 5 first

# Prediction = weights · features
predicted_mean = np.dot(mean_w, features)    # single number in days

```

Keeping data clean — the validator

Before any data reaches the models, it passes through a gatekeeper. The gatekeeper's job is to catch data so implausible that it would corrupt the model if accepted at face value.

Why validate at all? A user who taps the wrong date could submit a 3-day cycle. A user who stops using the app for 2 months could submit a 65-day cycle. A phone with the wrong clock could submit future dates. All of these would shift the model in the wrong direction if accepted.

What gets checked

1 Cycle length range

Currently 21–40 days. Cycles outside this are rejected with an error. The product brief recommends widening this to 14–90 days and accepting unusual cycles with reduced influence rather than rejecting them outright.

2 Period length range

Must be 2–10 days. A 1-day period is almost certainly a logging error. More than 10 days would be medically unusual rather than normal tracking data.

3 Quality flags

Each cycle receives a label: NORMAL, OUTLIER (more than 3 SDs from this user's mean), SUSPICIOUS (more than 4 SDs), INCOMPLETE, or IRREGULAR. Currently all passing cycles receive full weight regardless of flag. Adding weight-based handling is on the priority list.

4 Date plausibility

Start dates cannot be in the future. Cycles cannot end before they start. These catch obvious clock or entry errors.

Where validation sits in the flow



Current limitation: The validator either accepts a cycle fully or rejects it. The brief recommends a middle ground — accept unusual cycles but give them less influence. This requires adding a weight parameter to the model's update function. It is the highest-priority item for the next development sprint.

How a prediction request travels through the system

When the Oky app asks "what is this user's next predicted cycle?", here is exactly what happens — step by step.

1 App sends a POST request

The request contains: user ID, recent cycle history (list of lengths in days), age, and optionally days since menarche. Sent as JSON over HTTPS.

2 API Gateway receives it

AWS API Gateway is the front door. It handles HTTPS and routes the request to the right Lambda function. It does not inspect the data — that is the Lambda's job.

3 Lambda runs the inference function

The inference Lambda parses the JSON, validates the data, then loads this user's saved model state from DynamoDB.

4 Choose which model to use

$n_samples < 3 \rightarrow$ baseline model. $n_samples \geq 3 \rightarrow$ Bayesian model. The loaded model state is passed directly to the chosen model.

5 Return the prediction

The model returns: predicted cycle length, lower bound, upper bound, confidence score. The Lambda wraps this with the user ID, an encouragement message, and a consistency score, then returns it as JSON.

Example request and response

```
POST /dev/prediction
{
  "user_id": "abc123",
  "cycle_history": [28, 29, 27, 30],
  "age": 16,
  "days_since_menarche": 900
}

← 200 OK
{
  "user_id": "abc123",
  "prediction": {
    "predicted_cycle_length": 28.4,
    "lower_bound": 25.8,
    "upper_bound": 31.0,
    "confidence_score": 0.74
  },
  "encouragement_message": "Great job tracking!",
  "tracking_consistency_score": 0.87
}
```

Prediction does not update the model. The inference function is read-only. Updating (learning from a confirmed cycle) happens through a completely separate flow: confirm → update Lambda via SQS queue → DynamoDB write. This separation means predictions are always fast, even during a background update.

SECTION 8

The cloud plumbing — AWS services

The engine runs entirely on AWS. There are no servers to manage — each piece turns on when needed and turns off when idle. Here is what each service does and why it was chosen.

API Gateway — the front door

Sits in front of everything. Handles HTTPS encryption, URL routing, and CORS headers. You define which URL paths exist (`/prediction` , `/update` , `/sync`) and which Lambda each path calls. No application code needed here — pure configuration in `template.yaml` .

Lambda — the brains

Three separate functions, each doing one job:

FUNCTION	TRIGGER	WHAT IT DOES
<code>inference_function.py</code>	Every prediction request	Reads user model state, runs prediction, returns result. Fast and read-only.
<code>update_function.py</code>	SQS queue message	Receives a confirmed cycle from the queue, runs the Bayesian update, writes new model state to DynamoDB. Asynchronous — user does not wait for it.
<code>sync_function.py</code>	POST /sync from app	Accepts a bulk export from an offline device. Applies missed updates. Returns latest model state so the app can update its local copy.

DynamoDB — the memory

A fast key-value database. Two tables: one stores each user's model state (the weights and uncertainty grid the Bayesian model needs), the other stores raw cycle history. Both indexed by `user_id` . No servers to manage — billed only for actual reads and writes (`PAY_PER_REQUEST`).

SQS (queue) — the buffer

When the app sends a confirmed cycle, the inference Lambda does not update the model itself. It drops a message into a FIFO queue instead. The update Lambda picks that message up in the background. This keeps predictions fast and ensures updates for the same user are processed in order.

Everything is defined in `template.yaml` — the AWS SAM configuration file. This single file describes all three Lambdas, the API Gateway routes, both DynamoDB tables, and the SQS queue. Running `sam deploy` creates or updates all of it automatically.

The offline version

A significant portion of Oky's users are in areas with unreliable internet. The app must be fully functional without a connection. This means the entire predict-and-learn loop must run on the device itself.

The key design decision: Rather than writing different code for online and offline, the core logic (baseline model, Bayesian model, validator) is written once. The offline engine is a wrapper that uses local file storage instead of DynamoDB, and a local pending-updates file instead of SQS.

How the offline engine works

1 Same models, different storage

The offline engine imports the same `baseline_model.py` and `bayesian_model.py`. Instead of reading from DynamoDB, it reads from a JSON file on the device. Instead of writing to DynamoDB, it writes to that same local file.

2 Updates queue locally

When the user confirms a cycle offline, the update is saved to a local pending-updates list. When the device reconnects, those updates are bundled and sent to the sync endpoint.

3 Sync resolves conflicts

If the user used the app both offline and on another device while online, the sync function merges the two model states. The rule: the model with more confirmed cycles wins — more data means more accurate.

4 Export and import

A user can export their model state as a JSON blob and import it on a new device. This is also the format used for cloud sync — one format serves both purposes.

The sync format

```
{
  "model_state": {
    "user_id":    "abc123",
    "n_samples":  7,
    "mean_w":     [28.1, 0.9, -0.3, 0.2, 0.05],
    "cov_w":      [[0.4, 0.01, ...], [0.01, 0.02, ...], ...],
    "alpha":      1.0,
    "beta":       1.0
  },
  "cycle_history": [ ... list of raw cycle records ... ],
  "export_timestamp": "2026-06-26T10:30:00Z"
}
```

The offline engine is the best integration test. If offline predict → update → export → import → predict works correctly from end to end, your core logic is solid. The cloud layer is then just a matter of replacing file I/O with DynamoDB calls.

Build order — how to approach this from scratch

If you were starting this project over, this is the order that makes sense. Each layer depends on the one before it, so building in this order means you can test as you go and catch problems early.

1 Define the data model first

Write the Python dataclasses: `CycleData`, `BayesianModelState`, `PredictionResult`.

Getting these right first means every other piece of code knows what shape to expect. Changes here ripple everywhere, so it is worth getting right before writing logic.

2 Build the validator

Write `data_validation.py` with range checks and quality flags. Write tests for every edge case (too short, too long, future dates). This is the easiest module to test in isolation — pure input/output, no state.

3 Build the baseline model

Write `baseline_model.py` with the blending logic. Test it for 0, 1, and 2 cycles. Verify the weights (33/67 then 50/50) and that outputs clip to [21, 40]. Straightforward to verify by hand.

4 Build the Bayesian model

Write `bayesian_model.py` with `predict()` and `update()`. Test that the range narrows with consistent cycles and widens with variable ones. This is the hardest module — spend time on it before moving to infrastructure.

5 Build the offline engine

Wire models + validator together in `offline_engine.py` with local file storage. A working offline predict → update → predict loop means your core logic is proven before you touch any cloud services.

6 Write the Lambda functions

Wrap the working core in Lambda handlers. Start with `inference_function.py` (read-only, simplest). Then `update_function.py`. Then `sync_function.py`. Each handler is mostly: parse request → call core logic → format response.

7 Write template.yaml

Define all AWS resources: three Lambdas, API Gateway routes, two DynamoDB tables, the SQS FIFO queue. Run `sam local start-api` to test locally before deploying to the cloud.

8 Deploy and run live tests

Run `sam deploy` to create everything on AWS. Then run the automated live tests against the deployed endpoint to verify the full round trip: app → API Gateway → Lambda → DynamoDB → response.

Tests are in `tests/`. Run `pytest tests/` at any point to check core logic. The integration tests in `test_oky_app_integration.py` use mocked AWS services so they run without any real cloud connection. The offline engine tests in particular catch the vast majority of logic bugs before any cloud deployment is needed.

Applying this to a new app: Steps 1–5 (data model, validator, baseline, Bayesian, offline engine) are entirely reusable. The core prediction logic has no dependency on AWS — swap in any database and any deployment platform by replacing only the storage and Lambda layers. The learning algorithm itself travels unchanged.